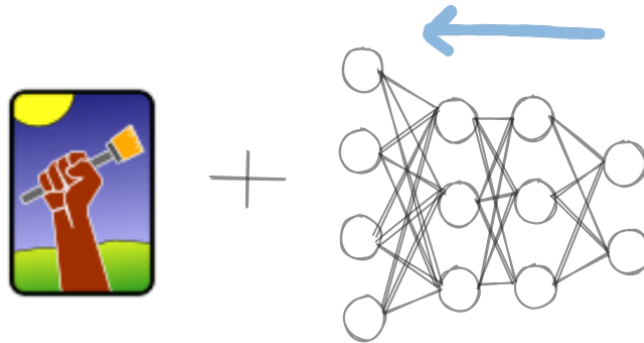


shallow2deep workflow in ilastik



Theodoros Katzalis

10/2024

Contents

1	Introduction	3
2	Ecosystem	4
3	Supervised training in ilastik	5
4	Deep learning models to use	5
4.1	Framework	5
5	Hyperparameter turning	5
6	Tiling training	6
6.1	Memory monitoring	7
7	Pre-processing	7
8	Dataloading	7
9	Post-processing	7

10	Logging	7
11	Segmentation masks	7
12	Current state of tiktorch	8
13	Architecture	8
14	Remote server requirements	8
15	Tiktorch	8
15.1	API	9
15.1.1	Start	9
15.1.2	Pause	9
15.1.3	Resume	9
15.1.4	Abort	9
15.1.5	Clean	9
16	Lazyflow	9
17	Ilastik view	9
18	Validation ownership	10
19	Next steps	10

1 Introduction

Ilastik is a desktop application that bridges the gap between machine learning and out of the box functionality intended for the bioimaging domain. For more about ilastik, check the documentation at <https://ilastik.github.io/>.

The main takeaway of the currently supported functionality is that training is only possible with shallow models (e.g. random forests), and neural network workflows are used only for inference. Our endeavor would be to integrate neural network training in ilastik.

The main motivation of this work originates by the shallow to deep project [1]. Its core idea is that supervised domain adaptation can achieve greater performance by incorporating a shallow model as a preliminary step. Supervised domain adaptation could be split to 1) train a model on a source domain given annotated datasets, and then 2) transfer it to a target one, while aiming to mitigate the domain shift.

Regarding the first part, the learning of the mapping from raw data to the ground truth segmentations via a deep model, is replaced by a shallow one, a random forest. Then a deep model learns to map the output of the shallow model (probability maps) to the ground truth segmentations (enhancer).

Regarding knowledge transfer to a target dataset, a random forest is initially trained utilizing ilastik’s functionality by providing a few annotations. Then, the pretrained enhancer attempts to correct the result of the new trained shallow model on the target domain. Subsequently, a deep learning model (segmentor) is trained for semantic segmentation from raw images, but this time using the output of the pretrained enhancer as pseudo labels. The whole workflow can be illustrated in the following figure:

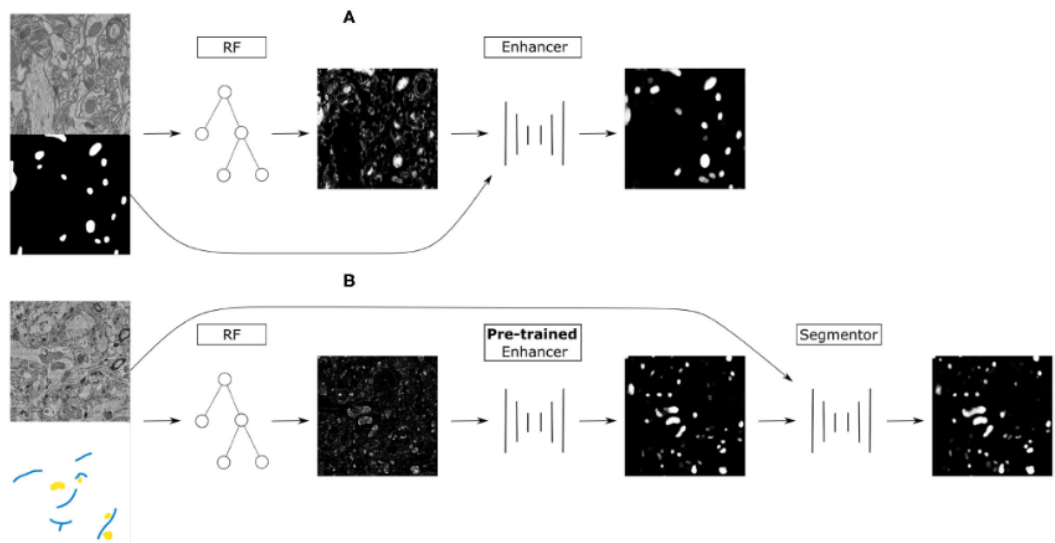


FIGURE 1 | (A) Training on the source dataset: many Random Forests are trained by subsampling patches of raw data and dense groundtruth segmentation. Random Forest predictions are used as inputs and groundtruth segmentation as labels to train the Prediction Enhancer CNN to improve RF segmentations. **(B)** Domain adaptation to the target dataset: a RF is trained interactively with brushstroke labels. The pre-trained PE is applied to improve the RF predictions. Optionally, PE predictions are used as pseudo-labels to train a segmentation network for even better results with no additional annotations, but using a larger computational budget.

Figure 1: Shallow to deep [1]

Currently, ilastik provides the above workflow, but it is missing the segmentor part. The goal is to implement the feature of training the segmentor. This will serve as the starting point for more sophisticated training workflows in the future. For next steps, refer to [19 Next steps](#).

2 Ecosystem

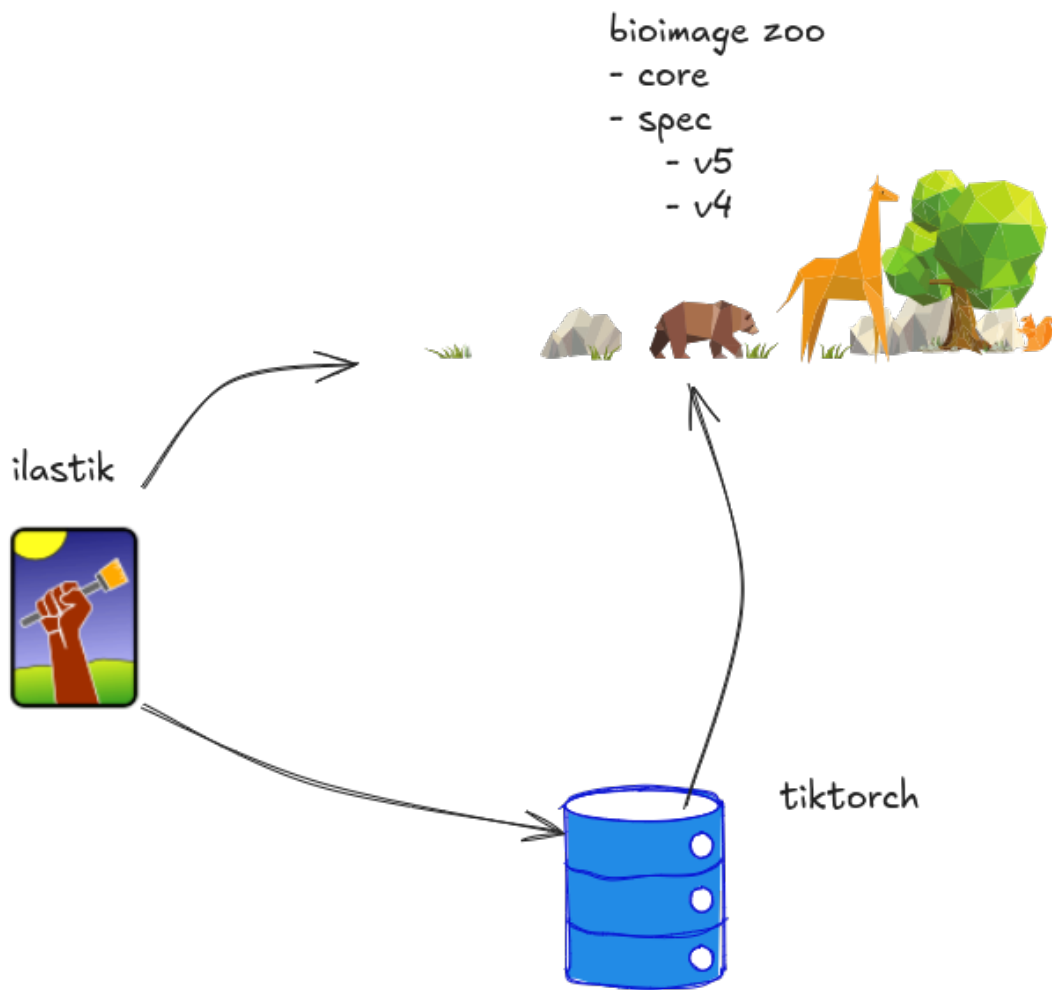


Figure 2: Ecosystem

The BioImage Model Zoo provides the infrastructure to specify, store, and retrieve **trained** deep learning models for the bioimage community. It should be emphasized that the models are trained, and the intended usage of them is only for inference. Currently, there are two fundamental packages, the spec-bioimage-io (<https://github.com/bioimage-io/spec-bioimage-io>), and the core-bioimage-io-python (<https://github.com/bioimage-io/core-bioimage-io-python>). The first one defines a specification that describes a deep learning model (e.g. type of inputs, outputs, weights, documentation), and provides the functionality of loading and saving it. The second one is mainly responsible to run a model (e.g. inference with tiling). For more you can refer to the publication [2].

Tiktorch (<https://github.com/ilastik/tiktorch>) attempts to manage the heavy load and orchestration of running deep learning models. The idea is that the last one is a computationally expensive task, so it could be decoupled from the client to a more suitable infrastructure (remote server). Effectively, it leverages BioImage Model Zoo to do the inference and loading of the models. Tiktorch is our backend system, and ilastik the client.

Ilastik handles all the workflows related with shallow models. For the deep learning ones, it sends requests to tiktorch to run the models and fetch the predictions. To avoid unnecessary communication delays for invalid requests, ilastik does some initial validation using the BioImage Model Zoo. Regarding validation of requests refer to [18 Validation ownership](#).

3 Supervised training in ilastik

As we have seen in the [1 Introduction](#), integrating supervised training for the semantic segmentation task, for the scope of the current project, will be narrowed down to the specific use case of the domain adaptation workflow. Overall, standardizing and exposing deep learning training would be the ideal goal. For more, you can refer to the [19 Next steps](#).

The requirements are:

P0 - Must have

- **P0.0** ilastik must support 2d and 3d deep learning training for the segmentor training.
- **P0.1** ilastik must provide training locally or remotely with GPU support.
- **P0.2** ilastik must be able to render the progress of training. Rendering of the progress can be asynchronous.
- **P0.3** ilastik must be able to resume, pause, save, and cancel a training process.

4 Deep learning models to use

The models that are used for the enhancers can be found in [torch-em](#). It is a list of 2d, and 3d models for nuclei, mito, and boundaries/membranes segmentation. These are meant to be provided and used for inference to get the pseudolabels. This is already supported by ilastik's workflow of domain adaptation.

4.1 Framework

5 Hyperparameter turning

Each of the models mentioned [4 Deep learning models to use](#), they could have their own set of parameters to tune ¹. Hyperparameter tuning is a heavy computational task. We could have some fixed values, and **optionally** allowing for tuning them given a user defined range of parameters.

One challenge is what kind of parameters should be tuned, and what type of hyperparameter strategy should we follow (e.g. grid search, bayesian optimization, random search).

A list of hyperparameters could include:

- Epochs
- Batch size
- Optimizer (e.g. Adam, SGD)
- Learning rate
- Batch normalization
- Layer normalization

¹U-Net-ResNet variations will have the same type of parameters such as batch size, learning rate optimizer, but if we support models such as custom U-Net, a new set of parameters needs to be configured such as the number of layers, number of pooling layers etc.

- Dropout layers
- Type of padding
- Loss function
- Kernel size, stride
- Patch size (6 Tiling training)

6 Tiling training

As we have mentioned, our task would be semantic segmentation, and we could provide some generic model architectures [4 Deep learning models to use](#). Another challenge would be to handle an arbitrary dataset size.

Memory constraints may not allow for a model to be trained on full images. Thus, tiling based training needs to be supported. To determine if training should be based on tiles, we need to take into account the most crucial memory factors which are batch size, image shape (2d or 3d), and model's complexity (e.g. number of feature maps).

For the option of tiling based training, we need to configure the halo, and the patch size. These should be exposed to the user with sensible defaults.

Should we do inference on a patch that it is different from the one in the training?

Currently, Ilastik has some hard-coded values for the patch size for 3d and 2d images. This is just a heuristic, and we aim to improve this. The same problem applies for the training as well.

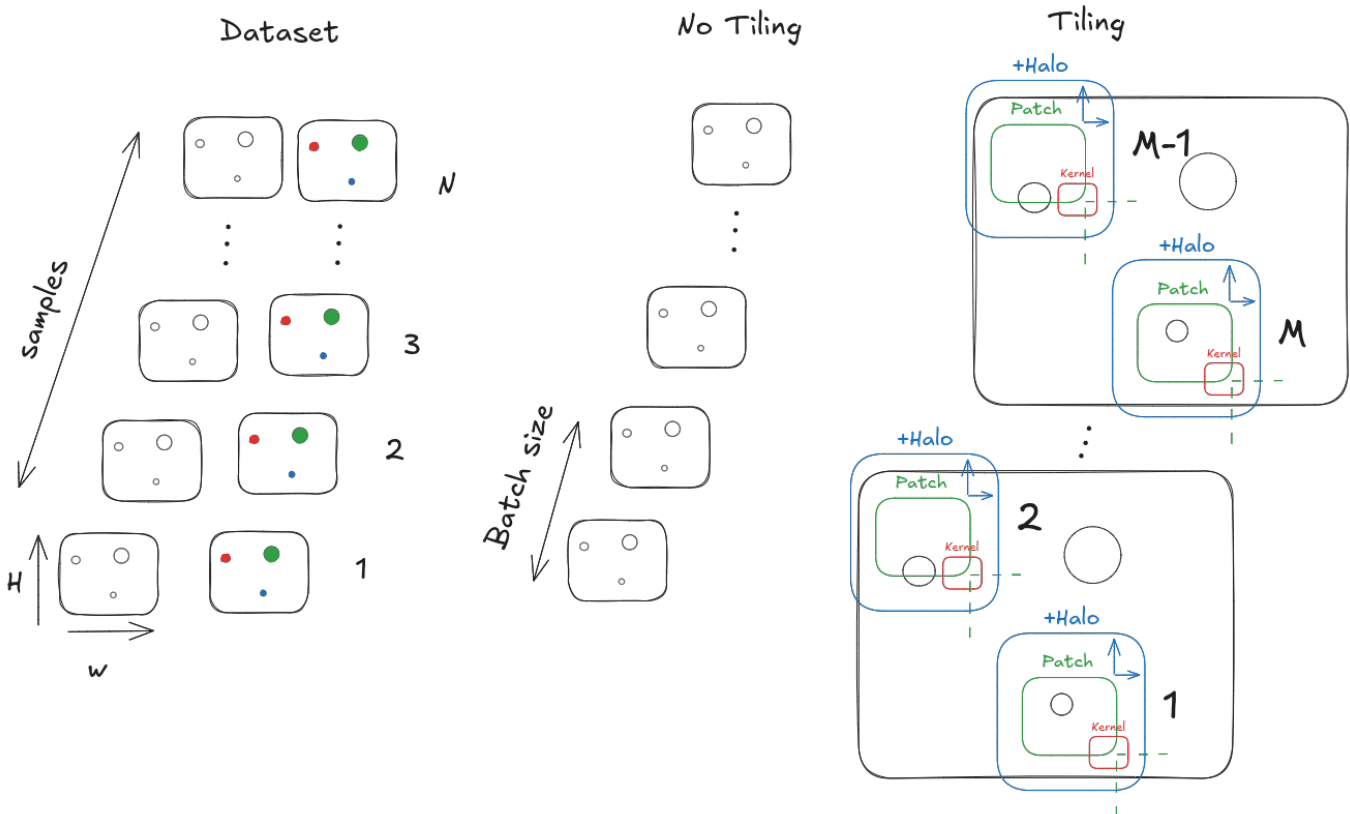


Figure 3: Demonstration of tiling for a 2d dataset. Patch $\equiv$ Tile

6.1 Memory monitoring

Memory constraints are referred to the GPU usage. We assume that for neural network training a GPU is required. Thus, we should possibly monitor the most impactful parameters that determine the memory usage.

Make a flowchart. What would be our memory parameter to decide about if my model fit to memory? This is an open-ended question. We could somehow monitor memory usage and suggest the user to reduce batch size, smaller tiles, or change model complexity? Or we just catch memory exceeded limit errors.

7 Pre-processing

- Downscaling
- Augmentations
- Oversampling
- Normalization

8 Dataloading

Dataloader questions:

- Do you shuffle the data?
- How do you handle when $\frac{\text{NumSamples}}{\text{BatchSize}} \% 1 \neq 0$? Drop last batch? Repeat samples (oversampling)?
- For the tiling the batch should be filled across images or within image?

9 Post-processing

What does bioimage core for post-processing?

10 Logging

To monitor the progress of the training, we need a logging framework. There are two well knowing frameworks used in the community, 1) the weights & biases and 2) tensorboard.

As we have seen, we will rely on existing machine learning logging frameworks. More specifically, we will assume that the logs are saved on the disk and Ilastik is able to read from that.

11 Segmentation masks

- Boundary based segmentation masks
- Semantic
- Instance

12 Current state of tiktorch

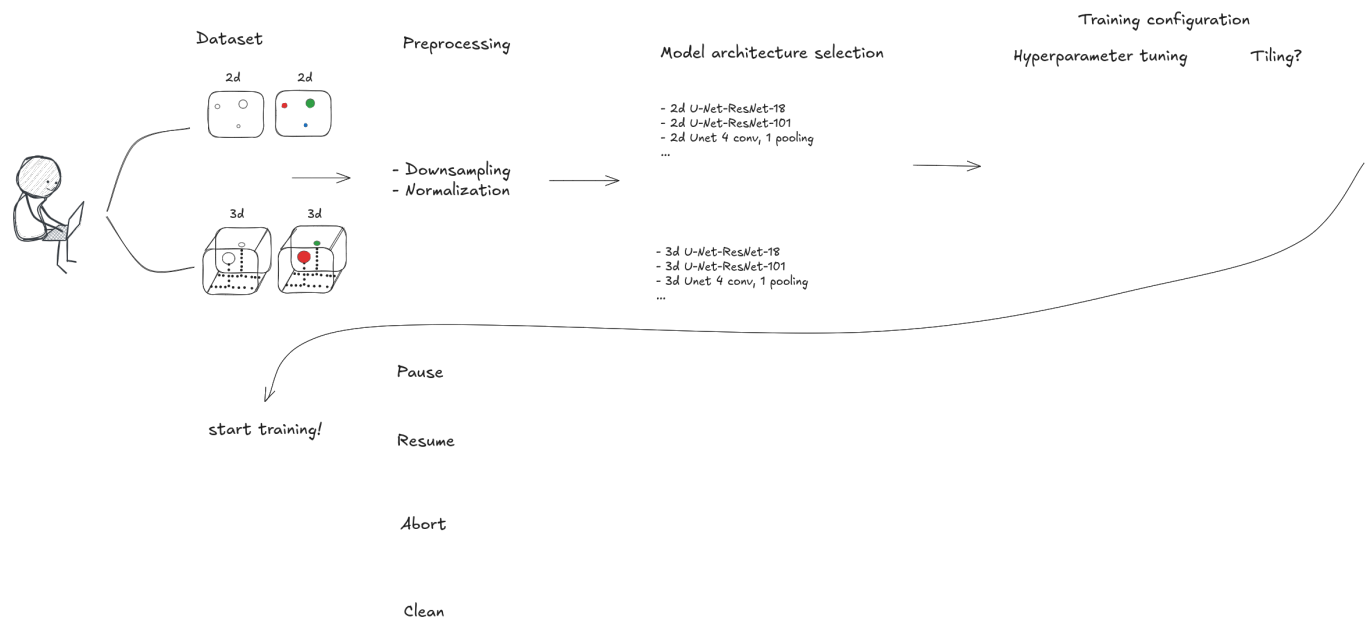
As we have seen tiktorch is our backend system to run the heavy load of neural networks. It supports inference of neural networks using the BioImage Model Zoo. Let's see what is the current state of it.

producer-consumer

threads, processes, orchestration

13 Architecture

High level diagram of what user will do. Provides input data, and ground truth.



Considerations:

- What channel to use for segmentation?

14 Remote server requirements

As we have seen we need to be able to save logs and the checkpoints. So apart from GPU resources needed for training, we need to have write access to the disk of the remote server. Alternatively, if this isn't possible, then the logs and the checkpoints should be streamed over grpc.

15 Tiktorch

Orchestration of resources for training. A gpu should be used for one model training. Is it possible to train two models with the same gpu? It is a good practise to do so?

15.1 API

- Start training
- Pause training
- Resume training
- Abort training
- Clean

15.1.1 Start

The start training should be responsible to initialize the model with the training parameters. This can include if hyperparameter tuning should be included or not.

15.1.2 Pause

Save training progress. Should each resume operation to update the last one? Or save to a separate file? Where the weights should be saved?

15.1.3 Resume

Load saved training progress, and continue training.

15.1.4 Abort

Abort the training process, release resources and remove any logs.

15.1.5 Clean

This call should be responsible to remove any artifacts (e.g. logs) created during training.

16 Lazyflow

What kind of operators to use.

17 Ilastik view

How it would look like?

18 Validation ownership

Validation of requests is crucial especially over networks that introduce significant latency. For example attempting to request to run a model, realizing that it doesn't exist in the server side, is a relative slow feedback loop. It can happen from the client side to speedup the process.

For the use cases of requests from ilastik to tiktorch, currently both of them perform some validation checks. On the other hand, bioimage model zoo performs some of them as well. In more detail:

- item1
- item2

It would probably be wise to standardize and modularize the validation. Each component should be able to perform validation, but the logic could be encapsulated in a central location. Bioimage model zoo should probably be the best candidate for that.

19 Next steps

One of the major bottlenecks with supervised machine learning, and especially in the biological domain, is the very expensive and time-consuming process of data annotation. Thus unsupervised learning and transfer learning are great candidates for our use case. Some next ideas could include:

1. Ilastik should be able to detach from a training process and reattach.
2. The user should be able to provide their own model definition written in PyTorch or Tensorflow.
3. BioImage Model Zoo integration.
4. Representation learning. Instead of trying to train from scratch a neural network in a supervised manner, would be very beneficial to make use of the latent space of representation learning methods.
5. Integrate already trained models (e.g. bioimage zoo), allowing finetuning to them.
6. Instance segmentation.

References

- [1] Alex Matskevych, Adrian Wolny, Constantin Pape, and Anna Kreshuk. From shallow to deep: exploiting feature-based classifiers for domain adaptation in semantic segmentation. *bioRxiv*, 2021.
- [2] Wei Ouyang, Fynn Beuttenmueller, Estibaliz Gómez-de Mariscal, Constantin Pape, Tom Burke, Carlos Garcia-López-de Haro, Craig Russell, Lucía Moya-Sans, Cristina de-la Torre-Gutiérrez, Deborah Schmidt, Dominik Kutra, Maksim Novikov, Martin Weigert, Uwe Schmidt, Peter Bankhead, Guillaume Jacquemet, Daniel Sage, Ricardo Henriques, Arrate Muñoz-Barrutia, Emma Lundberg, Florian Jug, and Anna Kreshuk. Bioimage model zoo: A community-driven resource for accessible deep learning in bioimage analysis. *bioRxiv*, 2022.